Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For

Linux Masterclass

Course Code: C579

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 1.0

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
1.0	1 October 2026	Initial release of the Linux Masterclass Learner Guide — step-by-step guides for all 17 KillerCoda Ubuntu labs across the four course topics, plus environment setup and a glossary.	Dr Alfred Ang

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

TABLE OF CONTENTS

Introduction **4**
Course Learning Outcomes **4**
Before You Start — Environment Setup **4**
Topic 01 — Linux Fundamental **5**
Lab 1 — Getting Started with the KillerCoda Ubuntu Playground 5
Lab 2 — Exploring the Linux File System 6
Lab 3 — Working with Files, Directories and Links 8
Lab 4 — Users, Ownership and Permissions 9
Topic 02 — Linux Shell Script **10**
Lab 5 — Customising the Shell Environment 11
Lab 6 — Redirection, Pipes and Text Processing 12
Lab 7 — Your First Shell Script 13
Lab 8 — User Input, Arguments and Arithmetic 14
Lab 9 — Conditionals — if, test and case 15
Lab 10 — Loops — for, while and until 17
Lab 11 — Functions, Exit Status and Debugging 18
Lab 12 — Automation Script — Backup with tar 19
Topic 03 — Linux Networking Configuration **20**
Lab 13 — Managing Processes and Scheduling with Cron 21
Lab 14 — Package Management and Web Server Deployment 22
Lab 15 — Network Configuration and Troubleshooting 23
Topic 04 — Linux Security **24**
Lab 16 — Hardening Linux Security 25
Lab 17 — Building a Security Audit Script 26
Revision and Further Practice **27**
Glossary **27**

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Introduction

This Learner Guide accompanies the course Linux Masterclass (C579), conducted by Tertiary Infotech Academy Pte Ltd. It provides detailed step-by-step instructions for all 17 hands-on labs, organised by the four course topics. Every lab is completed in a browser on the free KillerCoda Ubuntu playground — no installation is required on your own computer.

Use this guide alongside the course slides and the lab files in the labs/ folder of the course repository. The slides give each lab a briefing, its key commands and a verification check; this guide carries the full step-by-step walkthrough. Type the commands yourself rather than pasting them — the muscle memory is what makes the shell yours.

Course Learning Outcomes

- LO1: Analyze Linux components and coordinate on the Linux release.
- LO2: Configure Linux and select shell scripting for integrating software products.
- LO3: Execute tests on Linux networking and diagnose networking issues.
- LO4: Identify, modify and improve Linux security configuration.

Before You Start — Environment Setup

What you need

- A modern browser and a free KillerCoda account — the labs run on the KillerCoda Ubuntu Playground: https://killercoda.com/playgrounds/scenario/ubuntu
- The KillerCoda session gives you a single-node Ubuntu machine with root access via sudo; sessions are temporary, so re-clone the exercise files if you start a fresh session.
- The course exercise files, cloned from GitHub: https://github.com/tertiarycourses/C579-Linux-Masterclass
- No Linux installation on your own laptop is needed. If you prefer a local setup, VirtualBox + Ubuntu, WSL (Windows) or any cloud VM works identically.

Launch and verify your Ubuntu playground

Open the playground URL, sign in, and press Start. A terminal appears in the browser. Verify the environment and fetch the course files:

```
$ whoami                  # your user

$ uname -a                # Linux kernel release

$ lsb_release -a          # Ubuntu version
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
$ git clone https://github.com/tertiarycourses/C579-Linux-Masterclass.git
course

$ ls course/labs          # one folder per lab
```

Conventions used in every lab

- Commands are typed at the playground terminal; a leading sudo is shown where root is required.
- Lines starting with # inside code blocks are comments — do not type them.
- Each lab folder (labs/lab-NN-…) holds the scripts and data files that lab uses.
- Scripts you write go in ~/scripts; the labs build on one another, so do them in order.
- The KillerCoda session expires after a while — losing it is harmless; restart and re-clone.

Topic 01 — Linux Fundamental

Linux components · distributions · file system · users and permissions

Key concepts

- Linux OS & kernel — Linux is a free, open-source, Unix-like OS; the kernel plus apps and tools form a distribution.
- Distributions — RHEL, Fedora, CentOS, AlmaLinux, Ubuntu, Debian, SUSE — pick by need, support and package system.
- Ways to run Linux — Virtual machine, native install, live boot, cloud service, WSL — or a browser playground like KillerCoda.
- File system hierarchy — One tree from / — /bin, /etc, /home, /opt, /tmp, /usr, /var; everything is a file.
- Users & superuser — Regular, system, service and group accounts; root is all-powerful — use su/sudo with care.
- Permissions — Read/write/execute for user, group and other — set with chmod (alpha or octal), chown and umask.

Lab 1 — Getting Started with the KillerCoda Ubuntu Playground

Objective: Analyse the components of a Linux release — kernel, shell and distribution.

Goal: Launch the free browser-based KillerCoda Ubuntu playground, explore the terminal, and identify the components of the running Linux release: the kernel version, the distribution and the shell.

What you'll build

A working Ubuntu lab environment with the course exercise files cloned from GitHub. (Tools: KillerCoda Ubuntu playground, bash, git.)

Step-by-step

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

1. Open the KillerCoda Ubuntu playground in your browser and sign in (free account)

```
https://killercoda.com/playgrounds/scenario/ubuntu
```

2. Confirm who you are logged in as

```
whoami
```

3. Identify the kernel release and system architecture

```
uname -a
```

4. Identify the Ubuntu distribution and release

```
lsb_release -a
```

5. Inspect the distribution details recorded in the os-release file

```
cat /etc/os-release
```

6. Check which shell you are running

```
echo $SHELL
```

7. Clone the course exercise files from GitHub

```
git clone https://github.com/tertiarycourses/C579-Linux-Masterclass.git
course
```

8. List the lab folders you just downloaded

```
ls course/labs
```

9. Run the provided system-report script to summarise your Linux release components

```
bash course/labs/lab-01-getting-started/sysinfo.sh
```

Test it

uname -a shows a Linux kernel, lsb_release -a shows Ubuntu, and sysinfo.sh prints a system report with the kernel, distro, shell, uptime and disk usage.

Note: The scripts and data files for this lab are in labs/lab-01-*/ of the course repository. If a step fails, re-read its command for typos first, then check the lab folder's script against what you typed.

Lab 2 — Exploring the Linux File System

Objective: Analyse how a Linux system organises its release components on disk.

Goal: Navigate the Linux file system hierarchy from the root directory down: identify what lives in /bin, /etc, /home, /var and friends, use absolute and relative paths, and reveal hidden files.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

What you'll build

A mental map of the Linux directory tree and fluency with pwd, cd, ls and tree. (Tools: bash, coreutils, tree.)

Step-by-step

10. Print your current (home) directory

```
pwd
```

11. List the top of the file system hierarchy

```
ls /
```

12. Look at system configuration files in /etc

```
ls /etc | head
```

13. Look at log files in /var/log

```
ls /var/log
```

14. Change into /etc with an absolute path, then return home with the tilde shortcut

```
cd /etc && pwd && cd ~ && pwd
```

15. Move up one level with the .. shortcut

```
cd .. && pwd && cd ~
```

16. Show a long listing with file sizes in human-readable form

```
ls -lh /var/log
```

17. Show all files including hidden dot files in your home directory

```
ls -la ~
```

18. Install and use tree to visualise a directory tree two levels deep

```
sudo apt install -y tree && tree -d -L 2 /var
```

19. Identify the file type of a few files

```
file /bin/ls /etc/hostname /var/log
```

Test it

You can explain what /bin, /etc, /home, /tmp, /usr and /var hold, and ls -la ~ reveals hidden dot files such as .bashrc.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Note: The scripts and data files for this lab are in labs/lab-02-*/ of the course repository. If a step fails, re-read its command for typos first, then check the lab folder's script against what you typed.

Lab 3 — Working with Files, Directories and Links

Objective: Coordinate release files — create, copy, move, remove and link them.

Goal: Create and manage files and directories with touch, mkdir, cp, mv and rm; generate file sets with brace expansion; edit with nano; and compare hard links against symbolic links.

What you'll build

A working directory of files you created, renamed, moved and linked — including a hard link that survives deletion of its original and a symlink that breaks. (Tools: bash, coreutils, nano.)

Step-by-step

20. Make a working directory and change into it

```
mkdir ~/demo && cd ~/demo
```

21. Create an empty file with touch and check it

```
touch new_file.txt && ls -l
```

22. Create six files at once with brace expansion

```
touch apple_{A,B}{1..3}
```

23. Verify the expansion created apple_A1..apple_B3

```
ls apple_*
```

24. Edit new_file.txt with nano — type a line, then Ctrl-X, y, Enter to save

```
nano new_file.txt
```

25. Copy a file, then rename it with mv

```
cp new_file.txt copy.txt && mv copy.txt renamed.txt
```

26. Move the file into a subdirectory

```
mkdir sub && mv renamed.txt sub/
```

27. Copy the lorem ipsum exercise file into your demo directory

```
cp ~/course/labs/lab-03-files-directories/lorem_ipsum.txt lorem.txt
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

28. Create a hard link and a symbolic link to lorem.txt

```
ln lorem.txt lorem2 && ln -s lorem.txt lorem3
```

29. Delete the original and compare the links — the hard link still reads, the symlink is broken

```
rm lorem.txt && cat lorem2 | head -1 ; cat lorem3
```

30. Clean up the broken symlink and remove the directory tree when done

```
rm lorem3 && cd ~ && rm -r ~/demo
```

Test it

cat lorem2 still shows the text after lorem.txt is deleted, while cat lorem3 reports 'No such file or directory' — you can explain why.

Note: The scripts and data files for this lab are in labs/lab-03-*/ of the course repository. If a step fails, re-read its command for typos first, then check the lab folder's script against what you typed.

Lab 4 — Users, Ownership and Permissions

Objective: Coordinate access to release components with users, groups and permissions.

Goal: Work with the multiuser model: switch identities with su/sudo, read ls -l permission strings, and control access with chmod (alphabetic and octal), chown and umask.

What you'll build

A fruit.txt file whose permissions you set to rw-r--r-- and rwxrwxrw-, plus a file owned by root. (Tools: bash, sudo, coreutils.)

Step-by-step

31. Confirm your identity and list what sudo lets you do

```
whoami && sudo -l | head
```

32. Copy the fruit.txt exercise file to your home directory

```
cp ~/course/labs/lab-04-permissions/fruit.txt ~/fruit.txt && cd ~
```

33. Decode the permission string in a long listing

```
ls -l fruit.txt
```

34. Give user+group execute with alphabetic notation, then remove it

```
chmod ug+x fruit.txt && ls -l fruit.txt && chmod ug-x fruit.txt
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

35. Set exact permissions rw-/r--/r-- with alphabetic notation

```
chmod u=rw,g=r,o=r fruit.txt && ls -l fruit.txt
```

36. Set rwxrwxrw- using octal notation (776)

```
chmod 776 fruit.txt && ls -l fruit.txt
```

37. Change the file's owner to root, then confirm you can no longer write to it

```
sudo chown root fruit.txt && ls -l fruit.txt
```

38. Take ownership back with chown user:group

```
sudo chown $(whoami):$(whoami) fruit.txt
```

39. Display the current file-creation mask and predict a new file's permissions

```
umask
```

40. Create a file and verify the umask prediction (666 minus mask)

```
touch masked.txt && ls -l masked.txt
```

Test it

ls -l shows -rw-r--r-- after the alphabetic chmod and -rwxrwxrw- after chmod 776; the root-owned file refuses your write until you chown it back.

Note: The scripts and data files for this lab are in labs/lab-04-*/ of the course repository. If a step fails, re-read its command for typos first, then check the lab folder's script against what you typed.

Topic 02 — Linux Shell Script

Shell environment · text processing · Bash scripting: variables, conditionals, loops, functions

Key concepts

- The shell — A command-line interpreter (bash, zsh, ksh, csh…) — the default, scriptable interface to Linux.
- Environment — Aliases, environment variables, PATH and PS1 customise the shell; persist them in ~/.bashrc.
- Redirection & pipes — > >> < 2> redirect streams; | chains commands; /dev/null discards output.
- Text power tools — grep with regular expressions, sort, cut, uniq, head/tail, diff — process any text stream.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Bash scripts — A script = shebang + commands; make it executable with chmod +x and run it with ./script.sh.
- Script constructs — Variables, read input, positional arguments, arithmetic, if/test/case, for/while/until, functions, exit codes.

Lab 5 — Customising the Shell Environment

Objective: Configure the shell environment used to run integration scripts.

Goal: Configure your working environment: create aliases, set environment variables, extend PATH, customise the PS1 prompt, and persist the changes in ~/.bashrc.

What you'll build

A personalised shell with a cls alias, a custom prompt and an extended PATH that survives new sessions. (Tools: bash, nano, env.)

Step-by-step

41. List your current aliases

```
alias
```

42. Create aliases to fix a typo and clear the screen

```
alias grpe='grep' && alias cls='clear'
```

43. Try the new aliases, then remove one

```
cls && unalias grpe
```

44. Set an environment variable and read it back (no spaces around =)

```
MYNAME='Alfred' && echo $MYNAME
```

45. Export it so child processes inherit it

```
export MYNAME && bash -c 'echo $MYNAME'
```

46. Inspect the command search path

```
echo $PATH
```

47. Append a directory to PATH for this session

```
mkdir -p ~/bin && export PATH="${PATH}:$HOME/bin"
```

48. Customise the prompt to show user and directory

```
export PS1='\u@\h:\w$ '
```

49. Persist the alias, PATH and prompt by appending them to ~/.bashrc

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
echo "alias cls='clear'" >> ~/.bashrc && echo 'export
PATH="${PATH}:$HOME/bin"' >> ~/.bashrc
```

50. Reload the configuration and review your command history

```
source ~/.bashrc && history | tail
```

Test it

After source ~/.bashrc, cls clears the screen, echo $PATH ends with your ~/bin, and the prompt shows user@host:directory.

Note: The scripts and data files for this lab are in labs/lab-05-*/ of the course repository. If a step fails, re-read its command for typos first, then check the lab folder's script against what you typed.

Lab 6 — Redirection, Pipes and Text Processing

Objective: Select and combine text-processing tools for build and release pipelines.

Goal: Master the Unix I/O model: redirect stdout/stderr, pipe commands together, and process text with grep, regular expressions, sort, cut, uniq, wc, head, tail and diff.

What you'll build

A processed fruit list — sorted, de-duplicated and pattern-matched — plus a saved process listing. (Tools: bash, grep, sort, cut, uniq, wc, diff.)

Step-by-step

51. Change into the lab folder with the exercise files

```
cd ~/course/labs/lab-06-text-processing
```

52. Page through a long file with head and tail

```
head lorem_ipsum.txt && tail -3 lorem_ipsum.txt
```

53. Redirect a sorted copy of fruit.txt into a new file (overwrite with >)

```
sort fruit.txt > fruit_sorted.txt
```

54. Append a new fruit to the file with >>

```
echo 'mango' >> fruit_sorted.txt
```

55. Feed a file into a command with <

```
wc -l < fruit_sorted.txt
```

56. Pipe sort into uniq to drop duplicates

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
sort fruit.txt | uniq | head
```

57. Save the full process list to a file (old-deck activity)

```
ps aux > processes.txt
```

58. Search with grep: case-insensitive, whole word, count and invert

```
grep -i Apple fruit.txt && grep -w apple fruit.txt && grep -c apple
fruit.txt
```

59. Use regular expressions — anchors, classes and alternation

```
grep '^p' fruit.txt && grep 'berry$' fruit.txt && grep -E 'apple|pear'
fruit.txt
```

60. Find every fruit starting with s or p (old-deck activity)

```
grep '^[sp]' fruit.txt
```

61. Cut the 2nd field out of the presidents CSV and sort it uniquely

```
cut -d',' -f2 us_presidents.csv | sort -u | head
```

62. Compare two files with diff

```
diff original_file.txt revised_file.txt
```

63. Silence output you don't want with the null device

```
cat fruit.txt > /dev/null
```

Test it

grep -c apple fruit.txt returns a count, grep '^[sp]' lists only s/p fruits, processes.txt contains your ps aux snapshot, and the pipe chain prints unique sorted fruits.

Note: The scripts and data files for this lab are in labs/lab-06-*/ of the course repository. If a step fails, re-read its command for typos first, then check the lab folder's script against what you typed.

Lab 7 — Your First Shell Script

Objective: Select scripts as the tool for integrating and deploying software.

Goal: Write, save and execute your first Bash script: the #!/bin/bash shebang, echo output, variables, quoting rules and command substitution — then put it on your PATH.

What you'll build

hello.sh and vars.sh — executable scripts that greet the user and report the date via command substitution. (Tools: bash, nano, chmod.)

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Step-by-step

64. Create a scripts directory and change into it

```
mkdir -p ~/scripts && cd ~/scripts
```

65. Create hello.sh with the shebang and an echo line

```
printf '#!/bin/bash\necho "Hello World from $(whoami)"\n' > hello.sh
```

66. Try to run it — permission is denied because it is not executable

```
./hello.sh
```

67. Make it executable and run it again

```
chmod +x hello.sh && ./hello.sh
```

68. Create vars.sh using variables and quoting (single vs double quotes)

```
cp ~/course/labs/lab-07-first-script/vars.sh . && cat vars.sh
```

69. Run vars.sh and study how $a, $b and $c interpolate inside double quotes

```
bash vars.sh
```

70. Add command substitution — capture a command's output into a variable

```
echo "today=$(date +%F)" && cp ~/course/labs/lab-07-first-script/today.sh .
&& bash today.sh
```

71. Copy hello.sh into ~/bin so it runs from anywhere on your PATH

```
mkdir -p ~/bin && cp hello.sh ~/bin/ && hello.sh
```

Test it

./hello.sh prints the greeting with your username, vars.sh prints 'Hello, I have 16 apples', and hello.sh runs from any directory once it is in ~/bin.

Note: The scripts and data files for this lab are in labs/lab-07-*/ of the course repository. If a step fails, re-read its command for typos first, then check the lab folder's script against what you typed.

Lab 8 — User Input, Arguments and Arithmetic

Objective: Configure scripts that accept parameters for flexible deployment.

Goal: Make scripts interactive and reusable: read user input, use positional parameters $1..$9, $#, $@ and $0, and calculate with $(( )) arithmetic expansion.

What you'll build

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

greet.sh (reads your name), args.sh (echoes its arguments) and calc.sh (adds two numbers). (Tools: bash, read.)

Step-by-step

72. Change into your scripts directory and copy the lab scripts

```
cd ~/scripts && cp ~/course/labs/lab-08-input-arguments/*.sh .
```

73. Study greet.sh — read -p prompts the user and stores the answer

```
cat greet.sh
```

74. Run it and type your name when prompted

```
bash greet.sh
```

75. Study args.sh — $0 is the script, $1/$2 the arguments, $# the count, $@ all of them

```
cat args.sh
```

76. Run it with two arguments and check each expansion

```
bash args.sh apple banana
```

77. Run it with five arguments — what changes?

```
bash args.sh a b c d e
```

78. Study calc.sh — $(( )) performs integer arithmetic

```
cat calc.sh
```

79. Add 15 and 27 with calc.sh

```
bash calc.sh 15 27
```

80. Experiment with arithmetic expansion directly in the shell

```
echo $((7*6)) && x=10 && echo $((x+5))
```

Test it

greet.sh greets you by the name you typed, args.sh reports the right count with $#, and calc.sh 15 27 prints 42.

Note: The scripts and data files for this lab are in labs/lab-08-*/ of the course repository. If a step fails, re-read its command for typos first, then check the lab folder's script against what you typed.

Lab 9 — Conditionals — if, test and case

Objective: Configure scripts that branch on test results.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Goal: Branch your scripts: if/elif/else with test brackets for numeric, string and file checks, and the case statement for multi-way selection.

What you'll build

checknum.sh (positive/negative/zero), filecheck.sh (file tests) and fruitcase.sh (case menu). (Tools: bash, test.)

Step-by-step

81. Copy the lab scripts into your scripts folder

```
cd ~/scripts && cp ~/course/labs/lab-09-conditionals/*.sh .
```

82. Study checknum.sh — if / elif / else with numeric tests -gt, -lt, -eq

```
cat checknum.sh
```

83. Run it against different numbers

```
bash checknum.sh 5 && bash checknum.sh -3 && bash checknum.sh 0
```

84. Study filecheck.sh — file tests: -e exists, -f regular file, -d directory, -x executable

```
cat filecheck.sh
```

85. Test it against a file that exists and one that doesn't

```
bash filecheck.sh /etc/hostname && bash filecheck.sh /no/such/file
```

86. Compare strings with = and != inside [ ]

```
s='linux'; if [ "$s" = 'linux' ]; then echo match; fi
```

87. Study fruitcase.sh — case pattern matching with | alternatives and a *) default

```
cat fruitcase.sh
```

88. Run the case script with different fruits

```
bash fruitcase.sh apple && bash fruitcase.sh durian
```

Test it

checknum.sh classifies 5, -3 and 0 correctly; filecheck.sh reports /etc/hostname as an existing regular file; fruitcase.sh hits its default branch for durian.

Note: The scripts and data files for this lab are in labs/lab-09-*/ of the course repository. If a step fails, re-read its command for typos first, then check the lab folder's script against what you typed.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 10 — Loops — for, while and until

Objective: Configure scripts that repeat deployment steps over many items.

Goal: Automate repetition: for loops over lists, ranges and files; while and until loops with counters; and break/continue to control the flow.

What you'll build

forloop.sh, countdown.sh (while) and a renamer one-liner that processes a batch of files. (Tools: bash.)

Step-by-step

89. Copy the lab scripts

```
cd ~/scripts && cp ~/course/labs/lab-10-loops/*.sh .
```

90. Study forloop.sh — iterate a word list and a {1..5} range

```
cat forloop.sh
```

91. Run it

```
bash forloop.sh
```

92. Loop over files with a wildcard directly in the shell

```
for f in /etc/*.conf; do echo "config: $f"; done | head
```

93. Study countdown.sh — a while loop with a counter and ((i--))

```
cat countdown.sh
```

94. Run the countdown

```
bash countdown.sh 5
```

95. Study untilloop.sh — until runs UNTIL the condition becomes true

```
cat untilloop.sh
```

96. Run it and compare with while

```
bash untilloop.sh
```

97. Use break and continue — skip 3, stop at 7

```
for i in {1..9}; do [ $i -eq 3 ] && continue; [ $i -eq 7 ] && break; echo
$i; done
```

98. Batch-create then batch-rename files with a loop

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
mkdir -p ~/batch && cd ~/batch && touch report_{1..5}.txt && for f in
report_*.txt; do mv "$f" "${f%.txt}.bak"; done && ls
```

Test it

countdown.sh 5 counts 5-4-3-2-1 then 'Lift off!', the break/continue loop prints 1 2 4 5 6, and ~/batch holds report_1.bak … report_5.bak.

Note: The scripts and data files for this lab are in labs/lab-10-*/ of the course repository. If a step fails, re-read its command for typos first, then check the lab folder's script against what you typed.

Lab 11 — Functions, Exit Status and Debugging

Objective: Improve scripts with reusable functions and diagnose script issues.

Goal: Structure bigger scripts with functions, local variables and return values; read exit codes with $?; and debug with bash -x, set -x and trap.

What you'll build

funcs.sh (function library), exitcodes.sh and a debugged script traced with set -x. (Tools: bash, trap.)

Step-by-step

99. Copy the lab scripts

```
cd ~/scripts && cp ~/course/labs/lab-11-functions/*.sh .
```

100. Study funcs.sh — define a function, pass it arguments, keep variables local

```
cat funcs.sh
```

101. Run it and watch each function fire

```
bash funcs.sh
```

102. Check the exit status of a good and a bad command with $?

```
ls /etc > /dev/null; echo $? ; ls /nope 2>/dev/null; echo $?
```

103. Study exitcodes.sh — return values and exit N set the status your caller sees

```
cat exitcodes.sh && bash exitcodes.sh; echo "script exited with $?"
```

104. Trace a script line by line with bash -x

```
bash -x funcs.sh
```

105. Study debugme.sh — set -x switches tracing on mid-script, set +x off

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
cat debugme.sh && bash debugme.sh
```

106. Study trapit.sh — trap cleans up on exit or Ctrl-C

```
cat trapit.sh
```

107. Run trapit.sh and interrupt it with Ctrl-C to fire the trap

```
bash trapit.sh
```

Test it

funcs.sh prints output from each function, a failed ls returns a non-zero $?, bash -x shows each command prefixed with +, and Ctrl-C on trapit.sh prints the cleanup message.

Note: The scripts and data files for this lab are in labs/lab-11-*/ of the course repository. If a step fails, re-read its command for typos first, then check the lab folder's script against what you typed.

Lab 12 — Automation Script — Backup with tar

Objective: Select and configure a script that packages release files for deployment.

Goal: Put it all together: write backup.sh, a parameterised script that archives a directory with tar, gzip-compresses it, timestamps the filename, verifies the archive and reports disk usage.

What you'll build

backup.sh — a reusable backup tool producing dated .tar.gz archives with a verification listing. (Tools: bash, tar, gzip, du, df.)

Step-by-step

108. Create some files worth backing up

```
mkdir -p ~/project && touch ~/project/app_{1..5}.conf && echo data > ~/project/data.txt
```

109. Copy and study backup.sh — arguments, command substitution, tar czf, exit codes

```
cd ~/scripts && cp ~/course/labs/lab-12-backup-script/backup.sh . && cat backup.sh
```

110. Run it against the project directory

```
bash backup.sh ~/project
```

111. List the archive contents to verify the backup (tar tzf)

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
tar tzf ~/backups/project-*.tar.gz | head
```

112. Check the archive size and free disk space

```
du -h ~/backups && df -h / | tail -1
```

113. Extract the archive into a restore directory to prove it round-trips

```
mkdir -p ~/restore && tar xzf ~/backups/project-*.tar.gz -C ~/restore && ls
~/restore
```

114. Run backup.sh with no argument — the usage message and exit 1 protect the user

```
bash backup.sh; echo "exit=$?"
```

Test it

A dated project-YYYY-MM-DD.tar.gz appears in ~/backups, tar tzf lists the five .conf files, the restore directory round-trips them, and a missing argument exits with status 1.

Note: The scripts and data files for this lab are in labs/lab-12-*/ of the course repository. If a step fails, re-read its command for typos first, then check the lab folder's script against what you typed.

Topic 03 — Linux Networking Configuration

Processes and cron · package management · deploying services · network configuration tests

Key concepts

- Processes — ps, top and htop view processes; kill signals stop them; & , bg, fg and jobs manage background work.
- Cron scheduling — cron + crontab automate scripts on a schedule — minute, hour, day, month, weekday.
- Package managers — APT/dpkg on Debian-family, dnf/yum/rpm on Red Hat-family — install, upgrade, remove, autoremove.
- Deploying services — Install a web server (NGINX/Apache), manage it with systemctl, verify it with curl.
- Network configuration — ip addr, hostname, /etc/hosts, ping, ss — configure and inspect Linux networking.
- Remote access & transfer — SSH for secure shells; scp/sftp for secure file transfer; diagnose issues from test results.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 13 — Managing Processes and Scheduling with Cron

Objective: Execute configuration tests on running processes and schedule automated jobs.

Goal: Inspect and control processes with ps, top and kill; run jobs in the background with &, bg and fg; then automate a script with a cron schedule and verify it from its log.

What you'll build

A cron job that runs logdate.sh every minute and appends timestamps to ~/cron.log. (Tools: ps, top, kill, cron, crontab.)

Step-by-step

115. List your processes, then everything on the system

```
ps && ps aux | head
```

116. Watch processes interactively with top (press q to quit)

```
top -n 1 | head -15
```

117. Start a long-running background job with &

```
sleep 600 &
```

118. List jobs and find the sleep's process ID

```
jobs && ps aux | grep 'sleep 600' | grep -v grep
```

119. Kill the background process by PID (old-deck exercise)

```
kill $(pgrep -f 'sleep 600') && jobs
```

120. Open a second bash shell, find its PID with ps aux | grep bash, and kill it

```
bash & ps aux | grep bash | head && kill %+
```

121. Copy logdate.sh — a one-line script that appends the date to ~/cron.log

```
cp ~/course/labs/lab-13-processes-cron/logdate.sh ~/scripts/ && chmod +x
~/scripts/logdate.sh
```

122. Install a crontab entry that runs it every minute

```
( crontab -l 2>/dev/null; echo '* * * * * $HOME/scripts/logdate.sh' ) |
crontab -
```

123. Confirm the schedule is installed and cron is running

```
crontab -l && sudo service cron start
```

124. Wait a minute or two, then watch the log grow in real time

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
tail -f ~/cron.log
```

125. Remove the schedule when done

```
crontab -r
```

Test it

The killed sleep disappears from jobs, and ~/cron.log gains one timestamp line per minute while the crontab entry is active.

Note: The scripts and data files for this lab are in labs/lab-13-*/ of the course repository. If a step fails, re-read its command for typos first, then check the lab folder's script against what you typed.

Lab 14 — Package Management and Web Server Deployment

Objective: Configure software products and deploy a release to the platform, then test it.

Goal: Manage software with APT and dpkg, then deploy a real service: install NGINX, publish a page, control the service, and run a configuration test with curl to interpret the result.

What you'll build

A running NGINX web server serving your own index.html, verified by an HTTP 200 from curl. (Tools: apt, dpkg, nginx, systemctl/service, curl.)

Step-by-step

126. Refresh the package lists (always do this first)

```
sudo apt update
```

127. Search the repositories and inspect a package before installing

```
apt search nginx 2>/dev/null | head && apt show nginx | head
```

128. Install the NGINX web server

```
sudo apt install -y nginx
```

129. Confirm what dpkg installed

```
dpkg -l | grep nginx
```

130. Start the service and check its status

```
sudo service nginx start && service nginx status
```

131. Run a configuration test: request the default page and read the HTTP status

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
curl -I http://localhost
```

132. Publish your own page into the web root

```
echo '<h1>Deployed on Linux by '$(whoami)'</h1>' | sudo tee
/var/www/html/index.html
```

133. Re-test and interpret the result — the body should now be your page

```
curl http://localhost
```

134. Validate the NGINX configuration file syntax like a release engineer

```
sudo nginx -t
```

135. Practise the remove/clean cycle (then reinstall if you wish)

```
sudo apt remove -y nginx && sudo apt autoremove -y
```

Test it

curl -I returns HTTP/1.1 200 OK and curl http://localhost shows your deployed heading; nginx -t reports the configuration file syntax is ok.

Note: The scripts and data files for this lab are in labs/lab-14-*/ of the course repository. If a step fails, re-read its command for typos first, then check the lab folder's script against what you typed.

Lab 15 — Network Configuration and Troubleshooting

Objective: Execute network configuration tests and diagnose issues from the results.

Goal: Inspect and test Linux networking: interfaces and addresses with ip, name resolution with /etc/hosts, reachability with ping, sockets with ss — then run netcheck.sh to automate the diagnosis and interpret its report.

What you'll build

netcheck.sh — a network health-check script that tests interface, DNS, gateway and web reachability and prints a PASS/FAIL report. (Tools: ip, ping, ss, getent, curl, ssh/scp (concept).)

Step-by-step

136. List the network interfaces and their IP addresses

```
ip addr
```

137. Show the routing table and identify the default gateway

```
ip route
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

138. Check the hostname and its resolution

```
hostname && getent hosts $(hostname)
```

139. Test reachability with ping (4 packets)

```
ping -c 4 8.8.8.8
```

140. Test name resolution + reachability together

```
ping -c 4 google.com
```

141. Map a friendly name in /etc/hosts and ping it

```
echo '127.0.0.1 myapp.local' | sudo tee -a /etc/hosts && ping -c 2
myapp.local
```

142. List listening sockets and the services behind them

```
ss -tuln
```

143. Fetch a page over HTTP to prove layer-7 connectivity

```
curl -Is https://www.google.com | head -1
```

144. Copy and study netcheck.sh — each test maps to a diagnosis when it fails

```
cp ~/course/labs/lab-15-network-troubleshooting/netcheck.sh ~/scripts/ &&
cat ~/scripts/netcheck.sh
```

145. Run the health check and read the PASS/FAIL report

```
bash ~/scripts/netcheck.sh
```

146. Interpret: which failure would mean a DNS problem? A gateway problem? Discuss.

Test it

ping 8.8.8.8 and ping google.com both succeed, ss -tuln lists the listening services, and netcheck.sh prints PASS for interface, gateway, DNS and web tests.

Note: The scripts and data files for this lab are in labs/lab-15-*/ of the course repository. If a step fails, re-read its command for typos first, then check the lab folder's script against what you typed.

Topic 04 — Linux Security

Security principles · least privilege · firewall · hardening and audit scripts

Key concepts

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Security trade-offs — Nothing is perfectly secure — balance convenience against risk; assess severity, probability and cost.
- Multiuser model — Every file and process has an owner; compromising one account need not compromise the system.
- Least privilege — Use root only when required; avoid shared accounts and direct root logins; control access with sudo.
- Minimise & encrypt — Install only what you need; replace FTP/telnet/HTTP with SFTP/SSH/HTTPS.
- Firewall & monitoring — Filter connections with ufw/iptables; review logs and monitor system activity routinely.
- Continuous improvement — Audit configurations with scripts, identify improvements and implement modifications.

Lab 16 — Hardening Linux Security

Objective: Identify potential improvements to the Linux security configuration and implement them.

Goal: Apply the security principles: enforce least privilege on files, lock down SSH-style access, review accounts, and enable the ufw firewall so only the services you need are reachable.

What you'll build

A hardened lab machine — private key file at 600, firewall enabled with only SSH/HTTP allowed, and a reviewed account list. (Tools: chmod, passwd, ufw, ss, sudo.)

Step-by-step

147. Review the accounts on the system — real users have a shell, service accounts mostly don't

```
cut -d: -f1,7 /etc/passwd | head -15
```

148. Confirm only root has UID 0 (any other UID-0 user is a red flag)

```
awk -F: '$3==0 {print $1}' /etc/passwd
```

149. Create a sensitive file and apply least privilege — owner-only read/write

```
touch ~/secret.key && chmod 600 ~/secret.key && ls -l ~/secret.key
```

150. Find world-writable files in your home tree — each is a risk to fix

```
find ~ -maxdepth 2 -type f -perm -o+w 2>/dev/null
```

151. Check which services are listening before the firewall goes up

```
ss -tuln
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

152. Install and enable the ufw firewall, allowing only SSH and HTTP

```
sudo apt install -y ufw && sudo ufw allow ssh && sudo ufw allow http &&
sudo ufw --force enable
```

153. Verify the firewall rules

```
sudo ufw status verbose
```

154. Practise least privilege with sudo — run one privileged command, not a root shell

```
sudo tail -3 /var/log/dpkg.log
```

155. Discuss: which insecure services would you replace? (FTP→SFTP, telnet→SSH, HTTP→HTTPS)

Test it

ls -l shows -rw------- on secret.key, ufw status reports active with only 22 and 80 allowed, and the UID-0 check returns root alone.

Note: The scripts and data files for this lab are in labs/lab-16-*/ of the course repository. If a step fails, re-read its command for typos first, then check the lab folder's script against what you typed.

Lab 17 — Building a Security Audit Script

Objective: Implement a script that audits the configuration and surfaces improvements.

Goal: Combine everything you learned — functions, conditionals, loops, pipes and text processing — into audit.sh, a security audit script that checks accounts, permissions, listening services and disk usage, then writes a timestamped report.

What you'll build

audit.sh — a security audit tool producing ~/audit-YYYY-MM-DD.txt with a finding per check. (Tools: bash, awk, find, ss, df.)

Step-by-step

156. Copy and study audit.sh — one function per check, a report built with redirection

```
cp ~/course/labs/lab-17-security-audit/audit.sh ~/scripts/ && cat
~/scripts/audit.sh
```

157. Run the audit

```
bash ~/scripts/audit.sh
```

158. Read the report it generated

```
cat ~/audit-*.txt
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

159. Interpret each finding: UID-0 accounts, world-writable files, listening ports, disk usage
160. Fix a finding — remove world-write from any flagged file — and re-run the audit

```
chmod o-w <flagged-file> && bash ~/scripts/audit.sh
```

161. Extend the script: add a check that the ufw firewall is active (exit code of ufw status)

```
sudo ufw status | grep -q 'Status: active' && echo PASS || echo FAIL
```

162. Schedule the audit to run daily at 07:00 with cron — continuous improvement

```
( crontab -l 2>/dev/null; echo '0 7 * * * $HOME/scripts/audit.sh' ) |
crontab - && crontab -l
```

163. Remove the schedule when done

```
crontab -r
```

Test it

audit.sh writes a dated report listing each check with PASS/FAIL, the re-run shows your fix flipped a FAIL to PASS, and crontab -l shows the daily 07:00 entry before cleanup.

Note: The scripts and data files for this lab are in labs/lab-17-*/ of the course repository. If a step fails, re-read its command for typos first, then check the lab folder's script against what you typed.

Revision and Further Practice

- Redo every lab from memory until the commands come without looking.
- Review each lab's 'Test it' check: it states exactly what a correct configuration looks like.
- Sharpen your readiness on the Tertiary Infotech practice exams portal: https://exams.tertiaryinfotech.com
- Extend the scripts: add error handling to backup.sh, more checks to audit.sh, or new tests to netcheck.sh.
- Good free references: linuxcommand.org, the Bash manual (man bash), and the Ubuntu Server Guide.

Glossary

- **Kernel —** The core of the operating system — manages CPU, memory, devices and processes.
- **Distribution (distro) —** The Linux kernel packaged with libraries, utilities and applications (e.g. Ubuntu, RHEL).

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- **Shell —** A command-line interpreter (bash, zsh, ksh…) — the scriptable interface to Linux.
- **Shebang (#!) —** The first line of a script naming its interpreter, e.g. #!/bin/bash.
- **Environment variable —** A named value the shell and child processes can read, e.g. $PATH, $HOME.
- **PATH —** The ordered list of directories the shell searches to find a command.
- **Redirection —** Sending a command's input/output to files or other streams with < > >> 2>.
- **Pipe (|) —** Connecting one command's output to the next command's input.
- **Regular expression —** A pattern language for matching text, used by grep, sed and awk.
- **Exit status —** The 0–255 code a command returns; 0 means success, non-zero means failure ($?).
- **Positional parameters —** A script's arguments: $1, $2 …, with $# the count and $@ all of them.
- **cron / crontab —** The time-based job scheduler and the tool that edits your schedule.
- **Package manager —** Software that installs, upgrades and removes packages (apt/dpkg, dnf/rpm).
- **Permissions —** Read/write/execute rights for user, group and other, set with chmod/chown/umask.
- **root / superuser —** The all-powerful administrative account, borrowed safely via su and sudo.
- **Firewall (ufw) —** Filters network connections so only intended services are reachable.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg